

Competitive Programming

Section 27 Editorial: Bitmask DP

Competitive Programming Course

2026-08-06

Table of contents

1	Editorial overview	4
2	A. Skill Coverage	5
2.1	Restatement	5
2.2	Constraints and traps	5
2.3	Observations	5
2.4	Sample walk-through	5
2.5	Algorithm	6
2.6	Correctness proof	6
2.7	Complexity	6
2.8	C++ implementation notes	6
2.9	Python implementation notes	7
2.10	Full C++ solution	7
2.11	Full Python solution	8
2.12	Common wrong answers	9
2.13	Tests that matter	9
3	B. Assignment Profit	10
3.1	Restatement	10
3.2	Constraints and traps	10
3.3	Observations	10
3.4	Sample walk-through	10
3.5	Algorithm	10
3.6	Correctness proof	11
3.7	Complexity	11
3.8	Full C++ solution	11
3.9	Full Python solution	12
3.10	Common wrong answers	13
3.11	Tests that matter	13
4	C. Inspection Route	14
4.1	Restatement	14
4.2	Constraints and traps	14
4.3	Observations	14
4.4	Sample walk-through	14
4.5	Algorithm	15
4.6	Correctness proof	15
4.7	Complexity	15
4.8	Full C++ solution	16
4.9	Full Python solution	17
4.10	Common wrong answers	17
4.11	Tests that matter	18

5	D. Compatible Lineup	19
5.1	Restatement	19
5.2	Constraints and traps	19
5.3	Observations	19
5.4	Sample walk-through	20
5.5	Algorithm	20
5.6	Correctness proof	21
5.7	Complexity	21
5.8	C++ implementation notes	22
5.9	Python implementation notes	22
5.10	Full C++ solution	22
5.11	Full Python solution	23
5.12	Common wrong answers	24
5.13	Tests that matter	24
6	Testing discussion	25
7	What to carry forward	26

1 Editorial overview

All four problems use a mask to represent a subset of small objects.

Problem	Mask means	Extra state	Goal
A. Skill Coverage	skills already covered	none	minimum cost
B. Assignment Profit	jobs already used	none; worker is popcount	maximum profit
C. Inspection Route	locations already visited	current last location	minimum cycle cost
D. Compatible Lineup	selected performers	compatibility and score	maximum score

Reminder of operations:

```
test bit i:      (mask >> i) & 1
add bit i:      mask | (1 << i)
full mask:      (1 << n) - 1
number of bits: popcount(mask)
```

2 A. Skill Coverage

2.1 Restatement

Each candidate has a cost and covers some subset of the required skills. Choose some candidates with minimum total cost so every skill is covered. If no choice covers all skills, output -1 .

2.2 Constraints and traps

$n \leq 20$, so there are at most 2^{20} skill masks. With $m \leq 30$, enumerating candidate subsets would still be 2^m and impossible, while an $m * 2^n$ DP is the intended bound in both C++ and Python.

Costs can sum beyond 32-bit integer range. Use `long long` in C++.

The input uses skills numbered from 1; bits are easier with internal numbers from 0.

2.3 Observations

The exact identities of chosen candidates do not matter after we know:

```
which skills are currently covered
minimum cost paid to reach that covered set
```

When considering one candidate, we either skip them or take them. If we take them, the new covered set is the union of the old set and their skill set:

```
new_mask = mask | offer
```

2.4 Sample walk-through

The sample has four skills. Candidate 4 covers skills $\{1,3,4\}$ with cost 7, and candidate 5 covers skill $\{2\}$ with cost 3.

Their masks are:

```
candidate 4: 1101
candidate 5: 0010
union:      1111
cost:       10
```

No cheaper combination covers all four skills, so the answer is 10.

2.5 Algorithm

```
full = (1 << n) - 1
dp[0] = 0
dp[other masks] = INF

for each candidate with cost c and mask offer:
    next_dp = copy of dp
    for mask from 0 to full:
        merged = mask | offer
        next_dp[merged] = min(next_dp[merged], dp[mask] + c)
    dp = next_dp

if dp[full] is INF, print -1
else print dp[full]
```

2.6 Correctness proof

Invariant. After processing the first i candidates, $dp[mask]$ is the minimum cost to cover exactly the set of skills represented by $mask$ using only those i candidates.

Base case. Before processing any candidate, the only reachable covered set is the empty set with cost 0. Every non-empty set is unreachable, so INF is correct.

Induction step. Assume the invariant holds before candidate $i+1$. For any mask, a best solution after considering candidate $i+1$ either skips that candidate or takes that candidate.

If it skips the candidate, the cost and mask remain represented by the copied value in $next_dp[mask]$.

If it takes the candidate from an old covered set $mask$, the new covered set is $mask \mid offer$, and the cost is $dp[mask] + c$. The transition tries this for every old mask and keeps the minimum.

These two cases cover every possible valid choice involving candidate $i+1$, and no transition uses any unprocessed candidate. Therefore the invariant holds after candidate $i+1$.

At the end, $dp[full]$ is exactly the minimum cost to cover all skills, or unreachable if it is still INF . $\square$

2.7 Complexity

```
Time:  $O(m \cdot 2^n)$ 
Memory:  $O(2^n)$ 
```

2.8 C++ implementation notes

Convert each candidate to a mask while reading. Use a fresh vector for each candidate to preserve the “considered once” invariant.

2.9 Python implementation notes

Copy the list with `dp[:]` and skip INF states. Before the DP, coalesce equal skill masks to their cheapest cost and discard an offer that is no cheaper and covers a subset of another offer; such an offer can never improve an optimum. Python integers handle large costs automatically.

2.10 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    vector<pair<long long, int>> offers;
    for (int i = 0; i < m; i++) {
        long long cost;
        int k;
        cin >> cost >> k;
        int mask = 0;
        for (int j = 0; j < k; j++) {
            int skill;
            cin >> skill;
            mask |= 1 << (skill - 1);
        }
        offers.push_back({cost, mask});
    }

    int total = 1 << n;
    int full = total - 1;
    const long long INF = (1LL << 62);
    vector<long long> dp(total, INF);
    dp[0] = 0;

    for (auto [cost, offer] : offers) {
        vector<long long> next_dp = dp;
        for (int mask = 0; mask < total; mask++) {
            if (dp[mask] == INF) continue;
            int merged = mask | offer;
            next_dp[merged] = min(next_dp[merged], dp[mask] + cost);
        }
        dp.swap(next_dp);
    }

    cout << (dp[full] == INF ? -1 : dp[full]) << '\n';
}
```

```
    return 0;
}
```

2.11 Full Python solution

```
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    m = next(it)
    full = (1 << n) - 1
    offers: list[tuple[int, int]] = []
    for _ in range(m):
        cost = next(it)
        k = next(it)
        mask = 0
        for _ in range(k):
            skill = next(it) - 1
            mask |= 1 << skill
        offers.append((cost, mask))

    inf = 10**30
    dp = [inf] * (1 << n)
    dp[0] = 0
    for cost, offer in offers:
        new_dp = dp[:]
        for mask in range(1 << n):
            candidate = dp[mask] + cost
            merged = mask | offer
            if candidate < new_dp[merged]:
                new_dp[merged] = candidate
        dp = new_dp

    print(-1 if dp[full] >= inf else dp[full])

if __name__ == "__main__":
    main()
```

2.12 Common wrong answers

Using candidate subsets instead of skill masks.
Forgetting to subtract 1 from skill numbers.
Printing INF instead of -1 for impossible cases.
Using int in C++ when costs may be large.

2.13 Tests that matter

$n = 1$ with multiple candidates for the same skill.
Impossible coverage where one skill never appears.
One candidate covers every skill.
Many candidates have overlapping skill sets.

3 B. Assignment Profit

3.1 Restatement

Assign every worker to a distinct job. Worker i assigned to job j gives profit $a[i][j]$. Maximize the total profit.

3.2 Constraints and traps

$n \leq 18$, so $n!$ permutations are too many, but $n \cdot 2^n$ transitions are fine.

The state does not need to store both worker and mask. If `mask` contains k jobs, then exactly the first k workers have already been assigned.

3.3 Observations

For a mask of used jobs:

```
worker = popcount(mask)
```

The next action is to assign this worker to one unused job.

3.4 Sample walk-through

For the sample:

```
worker 0 -> job 0 gives 8  
worker 1 -> job 1 gives 9  
worker 2 -> job 2 gives 10
```

This uses every job once and gives total 27.

3.5 Algorithm

```

dp[0] = 0
dp[other masks] = -INF

for mask from 0 to full:
    worker = popcount(mask)
    for each job not in mask:
        next = mask | (1 << job)
        dp[next] = max(dp[next], dp[mask] + profit[worker][job])

print dp[full]

```

3.6 Correctness proof

Invariant. For every mask, once it is processed, `dp[mask]` is the maximum profit obtainable by assigning the first `popcount(mask)` workers to exactly the jobs in `mask`.

Base case. `dp[0] = 0`: no workers assigned and no jobs used. Other states start at negative infinity because they have not been reached.

Induction step. Suppose the invariant is true for a mask. The next worker is determined uniquely as `popcount(mask)`. Any valid assignment extending this partial assignment must choose one job not yet in the mask. The transition tries every such job, adds the corresponding profit, and updates the new mask.

Conversely, every transition assigns the correct next worker to an unused job, so it produces a valid partial assignment. Taking the maximum keeps the best profit for each mask.

When `mask = full`, all `n` jobs have been used and all `n` workers have been assigned, so `dp[full]` is the optimal answer. $\square$

3.7 Complexity

```

Time:  $O(n \cdot 2^n)$ 
Memory:  $O(2^n)$ 

```

3.8 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

```

```

vector<vector<long long>> profit(n, vector<long long>(n));
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) cin >> profit[i][j];
}

int total = 1 << n;
const long long NEG = -(1LL << 60);
vector<long long> dp(total, NEG);
dp[0] = 0;

for (int mask = 0; mask < total; mask++) {
    int worker = __builtin_popcount((unsigned)mask);
    if (worker == n || dp[mask] == NEG) continue;
    for (int job = 0; job < n; job++) {
        if ((mask & (1 << job)) == 0) {
            int nxt = mask | (1 << job);
            dp[nxt] = max(dp[nxt], dp[mask] + profit[worker][job]);
        }
    }
}

cout << dp[total - 1] << '\n';
return 0;
}

```

3.9 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    profit = [[next(it) for _ in range(n)] for _ in range(n)]
    total = 1 << n
    neg = -10**30
    dp = [neg] * total
    dp[0] = 0

    for mask in range(total):
        worker = mask.bit_count()
        if worker == n or dp[mask] == neg:
            continue
        for job in range(n):
            if not (mask >> job) & 1:

```

```

        nxt = mask | (1 << job)
        value = dp[mask] + profit[worker][job]
        if value > dp[nxt]:
            dp[nxt] = value

    print(dp[total - 1])

if __name__ == "__main__":
    main()

```

3.10 Common wrong answers

Using popcount after adding the job, assigning the wrong worker.
 Allowing the same job twice.
 Trying all permutations for $n = 18$.
 Using a minimum DP when the statement asks for maximum profit.

3.11 Tests that matter

$n = 1$.
 The best solution is not greedy by row maximum.
 Large profits that require 64-bit sums.
 Rows with ties, to ensure the DP handles multiple equivalent choices.

4 C. Inspection Route

4.1 Restatement

Start at location 1, visit every location exactly once, and return to location 1. Moving from i to j has cost $c[i][j]$, and costs may be asymmetric. Find the minimum route cost.

4.2 Constraints and traps

$n \leq 16$, which is small enough for $O(n^2 2^n)$.

The route cost depends on the current last location. A one-dimensional `dp[mask]` is insufficient.

Do not assume $c[i][j] = c[j][i]$.

Remember to add the final edge back to location 1.

4.3 Observations

After we know:

```
visited set mask
current last location
```

the best future route depends only on those two facts. The exact order used to reach `last` is already summarized by the minimum cost in the DP state.

4.4 Sample walk-through

One optimal sample route is:

```
1 -> 2 -> 4 -> 3 -> 1
```

Its cost is:

```
10 + 10 + 9 + 6 = 35
```

The DP reaches state $\{1,2,4\}$, `last=4` before adding city 3, then closes the cycle by returning to city 1.

4.5 Algorithm

Use zero-indexed locations internally.

```
dp[1 << 0][0] = 0
all other states = INF

for mask from 0 to full:
    if start bit is not in mask, continue
    for last from 0 to n-1:
        for each nxt not in mask:
            new_mask = mask | (1 << nxt)
            dp[new_mask][nxt] =
                min(dp[new_mask][nxt], dp[mask][last] + cost[last][nxt])

answer = min over last of dp[full][last] + cost[last][0]
```

4.6 Correctness proof

Invariant. $dp[mask][last]$ is the minimum cost of any path that starts at location 0, visits exactly the locations in `mask`, and ends at `last`.

Base case. The path that starts at 0, visits only 0, and ends at 0 has cost 0, so $dp[1][0] = 0$. No other state is initially reachable.

Induction step. Assume the invariant is true for all states with k visited locations. Consider a valid state $(mask, last)$ with k locations. Any path extension that visits one new location must choose a location `nxt` not in `mask`, pay $cost[last][nxt]$, and end at `nxt` with visited set $mask \mid (1 \ll nxt)$. The transition tries every such `nxt`.

Conversely, every transition appends exactly one unvisited location to a valid path, so it creates a valid path for the new state. Taking the minimum over all ways to reach a state gives the optimal value.

After all locations are visited, a complete tour must return from its final location `last` to 0. Minimizing $dp[full][last] + cost[last][0]$ over all possible last locations gives the optimal cycle. $\square$

4.7 Complexity

```
States:  $n \cdot 2^n$ 
Transitions per state: up to  $n$ 
Time:  $O(n^2 \cdot 2^n)$ 
Memory:  $O(n \cdot 2^n)$ 
```

4.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    vector<vector<long long>> cost(n, vector<long long>(n));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) cin >> cost[i][j];
    }

    int total = 1 << n;
    int full = total - 1;
    const long long INF = (1LL << 62);
    vector<vector<long long>> dp(total, vector<long long>(n, INF));
    dp[1][0] = 0;

    for (int mask = 0; mask < total; mask++) {
        if ((mask & 1) == 0) continue;
        for (int last = 0; last < n; last++) {
            if (dp[mask][last] == INF) continue;
            for (int nxt = 0; nxt < n; nxt++) {
                if ((mask & (1 << nxt)) == 0) {
                    int new_mask = mask | (1 << nxt);
                    dp[new_mask][nxt] = min(
                        dp[new_mask][nxt],
                        dp[mask][last] + cost[last][nxt]
                    );
                }
            }
        }
    }

    long long answer = INF;
    for (int last = 0; last < n; last++) {
        answer = min(answer, dp[full][last] + cost[last][0]);
    }
    cout << answer << '\n';
    return 0;
}
```

4.9 Full Python solution

```
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    cost = [[next(it) for _ in range(n)] for _ in range(n)]
    total = 1 << n
    full = total - 1
    inf = 10**30
    dp = [[inf] * n for _ in range(total)]
    dp[1][0] = 0

    for mask in range(total):
        if not (mask & 1):
            continue
        for last in range(n):
            cur = dp[mask][last]
            if cur >= inf:
                continue
            for nxt in range(n):
                if not (mask >> nxt) & 1:
                    new_mask = mask | (1 << nxt)
                    value = cur + cost[last][nxt]
                    if value < dp[new_mask][nxt]:
                        dp[new_mask][nxt] = value

    answer = min(dp[full][last] + cost[last][0] for last in range(n))
    print(answer)

if __name__ == "__main__":
    main()
```

4.10 Common wrong answers

Forgetting the last dimension and losing the current location.
Allowing location 0 to be revisited before the end.
Forgetting the final return edge to location 0.
Assuming symmetric costs.
Starting from every location when the problem fixes location 1.

4.11 Tests that matter

`n = 1` should output 0.

`n = 2` should output `c[0][1] + c[1][0]`.

Asymmetric matrices where reversing an edge changes the answer.

Cases where the cheapest outgoing edge greedily gives a bad tour.

5 D. Compatible Lineup

5.1 Restatement

Choose a subset of at most twenty performers. A listed incompatible pair may not be chosen together. Scores can be negative and choosing nobody is allowed, so the goal is the greatest total score among all compatible subsets, possibly zero.

This is the local blind exercise for the section. Its statement does not tell you what information to store for a subset; this editorial develops that state after the attempt.

5.2 Constraints and traps

The central constraint is $n \leq 20$. There are $2^{20} = 1,048,576$ possible subsets: too many to inspect each pair from scratch for every subset, but small enough to keep one or two arrays indexed by a mask.

Scores have magnitude up to 10^9 . A valid lineup can contain twenty positive performers, so its total can reach $2 \cdot 10^{10}$. Use `long long` in C++; Python integers already grow as needed.

An incompatible pair is undirected. If (u, v) is read, then choosing u forbids v and choosing v forbids u .

Finally, negative scores matter. They do not make a compatible subset invalid, but the empty lineup is legal and has score zero. Initializing the answer to a negative infinity would incorrectly force a performer into an all-negative instance.

5.3 Observations

Let `mask` represent the exact set of selected performers. A direct test of a mask could scan every input edge and reject it if both endpoints have their bits set. Repeating that scan for all masks costs $O(m \cdot 2^n)$. It fits only narrowly and does unnecessary repeated work.

Instead, obtain each non-empty mask from one smaller mask. Pick one set bit, call its performer `p`, and remove it:

```
bit      = mask & -mask
p        = index of bit
previous = mask ^ bit
```

Now the lineup represented by `mask` is compatible exactly when both facts are true:

1. `previous` was compatible;
2. none of `p`'s incompatible neighbours is in `previous`.

The total score has the same one-item recurrence:

```
score[mask] = score[previous] + w[p]
```

For fast neighbour tests, make `forbidden[p]` a mask containing every performer incompatible with `p`. Then condition 2 is simply:

```
(forbidden[p] & previous) == 0
```

This is a useful bitmask-DP pattern: the mask itself describes the whole choice, while one removed bit supplies a unique smaller state from which to maintain a property incrementally.

5.4 Sample walk-through

Number performers from zero internally. For the sample, the scores are `[8, 6, -3, 7]`, and performer 1 conflicts with both 0 and 3.

Consider the mask `1001`, which selects performers 0 and 3:

```
mask      = 1001
bit       = 0001          (performer 0)
previous  = 1000          (performer 3)
score     = score[1000] + 8 = 7 + 8 = 15
```

`previous` is compatible. Performer 0 is incompatible only with performer 1, and bit 1 is not set in `1000`, so the new subset is compatible.

In contrast, for `0011` we remove performer 0 and get `previous = 0010`. That previous mask selects performer 1; the intersection of performer 0's forbidden mask with `0010` is non-zero. Therefore `0011` is rejected. The best accepted total is 15.

5.5 Algorithm

```
read scores and build forbidden[p] for every performer p

compatible[0] = true
total[0] = 0
answer = 0

for mask from 1 to (1 << n) - 1:
    bit = lowest set bit of mask
    p = index of bit
    previous = mask without bit

    total[mask] = total[previous] + score[p]
    compatible[mask] = compatible[previous] and
                      (forbidden[p] & previous) == 0
```

```

if compatible[mask]:
    answer = max(answer, total[mask])

print answer

```

The recurrence always points to a numerically smaller mask, because removing a set bit lowers the integer. Therefore increasing mask order has already computed `previous`.

5.6 Correctness proof

We prove that the algorithm prints the maximum score of any compatible lineup.

Lemma 1. For every mask, `total[mask]` equals the sum of the scores of exactly the performers whose bits are set in `mask`.

Proof. The claim is true for mask 0, whose sum is zero. For a non-empty mask, the algorithm removes one selected performer `p` to form `previous` and adds `score[p]` to `total[previous]`. By induction on the number of set bits, `total[previous]` is the sum of every other selected performer, so the result is precisely the sum for `mask`. $\square$

Lemma 2. For every mask, `compatible[mask]` is true exactly when no incompatible pair has both endpoints selected by `mask`.

Proof. The empty mask is compatible, establishing the base case. Consider a non-empty `mask`, with removed performer `p` and `previous` as in the algorithm. By induction, `compatible[previous]` is true exactly when all pairs wholly inside `previous` are allowed. The only pairs newly relevant after adding `p` are pairs between `p` and a selected performer in `previous`. The bit intersection `forbidden[p] & previous` is zero exactly when no such pair exists. Thus the algorithm marks `mask` compatible exactly in the required case. $\square$

Theorem. The printed `answer` is the maximum total score of a compatible lineup.

Proof. Every possible lineup corresponds to exactly one mask, and every mask is processed once. By Lemma 2, the algorithm considers `total[mask]` for exactly the compatible lineups. By Lemma 1, that value is the lineup's actual total score. `answer` starts with the empty compatible lineup's score and takes the maximum over all other compatible masks, so it equals the required optimum. $\square$

5.7 Complexity

```

Time:    O(n + m + 2^n)
Memory:  O(2^n + n)

```

Building the incompatibility masks costs $O(m)$. Each subset then does a constant number of bit operations and array accesses.

5.8 C++ implementation notes

`__builtin_ctz(mask)` returns the number of trailing zero bits, which is the index of the least significant set bit when `mask` is non-zero. Call it only inside the non-empty-mask loop. `1 << n` is safe here because `n <= 20`, but the accumulated scores still need `long long`.

`vector<char>` is a compact, ordinary boolean-like array. It avoids the special proxy behavior of `vector<bool>` without changing the recurrence.

5.9 Python implementation notes

Python has no built-in trailing-zero operation, but `lowest_bit.bit_length() - 1` gives the index after computing `lowest_bit = mask & -mask`. A `bytearray` stores compatibility flags compactly; values 0 and 1 work directly in an `if` condition.

Use `not (forbidden[performer] & previous)` for the zero-intersection test. Parentheses keep the intended precedence clear.

5.10 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;

    vector<long long> score(n);
    for (long long& value : score) cin >> value;

    vector<int> forbidden(n, 0);
    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        --u;
        --v;
        forbidden[u] |= 1 << v;
        forbidden[v] |= 1 << u;
    }

    int total_masks = 1 << n;
    vector<long long> total(total_masks, 0);
    vector<char> compatible(total_masks, false);
    compatible[0] = true;
    long long answer = 0;
```

```

for (int mask = 1; mask < total_masks; ++mask) {
    int performer = __builtin_ctz(mask);
    int previous = mask ^ (1 << performer);
    total[mask] = total[previous] + score[performer];
    compatible[mask] = compatible[previous] &&
        ((forbidden[performer] & previous) == 0);
    if (compatible[mask]) answer = max(answer, total[mask]);
}

cout << answer << '\n';
return 0;
}

```

5.11 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return

    it = iter(data)
    n = next(it)
    m = next(it)
    score = [next(it) for _ in range(n)]
    forbidden = [0] * n
    for _ in range(m):
        u = next(it) - 1
        v = next(it) - 1
        forbidden[u] |= 1 << v
        forbidden[v] |= 1 << u

    total_masks = 1 << n
    total = [0] * total_masks
    compatible = bytearray(total_masks)
    compatible[0] = 1
    answer = 0

    for mask in range(1, total_masks):
        lowest_bit = mask & -mask
        performer = lowest_bit.bit_length() - 1
        previous = mask ^ lowest_bit
        total[mask] = total[previous] + score[performer]
        if compatible[previous] and not (forbidden[performer] & previous):
            compatible[mask] = 1
            if total[mask] > answer:

```

```
        answer = total[mask]

    print(answer)

if __name__ == "__main__":
    main()
```

5.12 Common wrong answers

Checking only whether the last performer has a conflict and forgetting whether the previous subset was already invalid.
Treating an incompatible pair as directed and setting only one forbidden bit.
Using int for totals, which overflows on twenty scores of 10^9 .
Starting answer at the smallest score instead of allowing the empty lineup.
Calling `__builtin_ctz(0)`, which is undefined in C++.

5.13 Tests that matter

All scores negative and no conflicts: answer must be 0.
A complete incompatibility graph: at most one performer can be chosen.
No conflicts and twenty scores of 10^9 : checks 64-bit accumulation and scale.
A conflict graph with disconnected components: compatible choices may combine performers from different components.

6 Testing discussion

The random generators intentionally use small n so that the expected outputs can be checked by brute force for assignment, route, and lineup problems. This matters: bitmask DP bugs often pass samples while failing on a single missing bit test.

For your own stress tests:

1. Generate small random cases.
2. Solve them with brute force over subsets or permutations.
3. Compare against the DP.
4. Only then trust the optimized constraints.

7 What to carry forward

Bitmask DP is a compact way to remember choices over a small set. The three most reusable states are:

```
dp[mask]           coverage or used-set optimization
dp[mask] with popcount(mask) deciding the next row
dp[mask][last]     route/order problems where current position matters
```

Future advanced sections will add subset iteration tricks, meet-in-the-middle, and transforms. The foundation remains the same: define the mask meaning before writing the transition.